

Chapter 30

RSA Digital Signature Peripheral (RSA_DS)

30.1 Overview

The RSA Digital Signature Peripheral (RSA_DS) is used to verify the authenticity and integrity of a message using a cryptographic algorithm. This can be used to validate a device's identity to a server or to check the integrity of a message.

ESP32-P4 includes an RSA Digital Signature Peripheral (RSA_DS) providing hardware acceleration of messages' signatures based on RSA. HMAC is used as the key derivation function (KDF) to output the *RSA_DS_KEY* using a key stored in eFuse or deployed by the Key Manager as the input key. Subsequently, the RSA_DS peripheral uses *RSA_DS_KEY* to decrypt the pre-encrypted parameters and calculate the signature. The whole process happens in hardware so that all the keys involved during the calculating process cannot be seen by users, guaranteeing the security of the operation.

30.2 Features

- RSA digital signatures with key length up to 4096 bits
- Encrypted private key data, only decryptable by the RSA_DS peripheral
- SHA-256 digest to protect private key data against tampering by an attacker

30.3 Functional Description

30.3.1 Overview

The RSA_DS peripheral calculates RSA signatures as $Z = X^Y \bmod M$, where Z is the signature, X is the input message, and Y and M are the RSA private key parameters.

Private key parameters are stored in flash as ciphertext. They are decrypted using *RSA_DS_KEY* which can:

- Be calculated by the RSA_DS peripheral via the HMAC peripheral. The required inputs (*HMAC_KEY*) to generate the key are only stored in eFuse and can only be accessed by the HMAC peripheral. That is to say, the RSA_DS peripheral hardware can decrypt the private key, and the private key in plaintext is never accessed by the software. For more detailed information about eFuse and HMAC peripherals, please refer to Chapter 8 [eFuse Controller \(EFUSE\)](#) and Chapter 27 [HMAC Accelerator \(HMAC\)](#).
- Deploy from Key Manager module. *RSA_DS_KEY* must be deployed by the Key Manager in a secured way.

The input message X will be sent directly to the RSA_DS peripheral by the software each time a signature is needed. After the RSA signature operation, the signature Z is read back by the software.

For better understanding, we define some symbols and functions here, which are only applicable to this chapter:

- 1^s : A bit string consisting of s bits with the value of “1”.
- $[x]_s$: A bit string of s bits, in which s is an integer multiple of 8 bits. If x is a number ($x < 2^s$), it is represented in little-endian byte order in the bit string. x may be a variable such as $[Y]_{4096}$ or a hexadecimal constant such as $[0x0C]_8$. If necessary, the value $[x]_t$ can be right-padded with $(s - t)$ number of zeros to reach s bits in length, and finally get $[x]_s$. For example, $[0x05]_8 = 00000101$, $[0x05]_{16} = 0000010100000000$, $[0x0005]_{16} = 0000000000000101$, $[0x13]_8 = 00010011$, $[0x13]_{16} = 0001001100000000$, $[0x0013]_{16} = 0000000000010011$.
- $||$: A bit string concatenation operator for joining multiple-bit strings into a longer bit string.

30.3.2 Private Key Operands

Private key operands Y (private key exponent) and M (key modulus) are generated by the user. They have a particular RSA key length (up to 4096 bits). Two additional private key operands are needed: $\bar{r}$ and M' . These two operands are derived from Y and M .

Operands Y , M , $\bar{r}$, and M' are encrypted by the user along with an authentication digest and stored as a single ciphertext C . C is input to the RSA_DS peripheral in this encrypted format, decrypted by the hardware, and then used for RSA signature calculation. A detailed description of how to generate C is provided in Section 30.3.3.

The RSA_DS peripheral supports RSA signature calculation $Z = X^Y \bmod M$, in which the length of operands should be $N = 32 \times x$ where $x \in \{1, 2, 3, \dots, 128\}$. The bit lengths of arguments Z , X , Y , M , and $\bar{r}$ should be an arbitrary value in N , and all of them in a calculation must be of the same length, while the bit length of M' should always be 32. For more detailed information about RSA calculation, please refer to Section 28.3.2 *Large-Number Modular Exponentiation* in Chapter 28 *RSA Accelerator (RSA)*.

30.3.3 Software Prerequisites

If you want to use the RSA_DS peripheral, the software needs a series of preparations, as shown in Figure 30.3-1. The left side lists preparations required by the software before the hardware starts the RSA signature calculation, while the right side lists the hardware workflow during the entire calculation procedure.

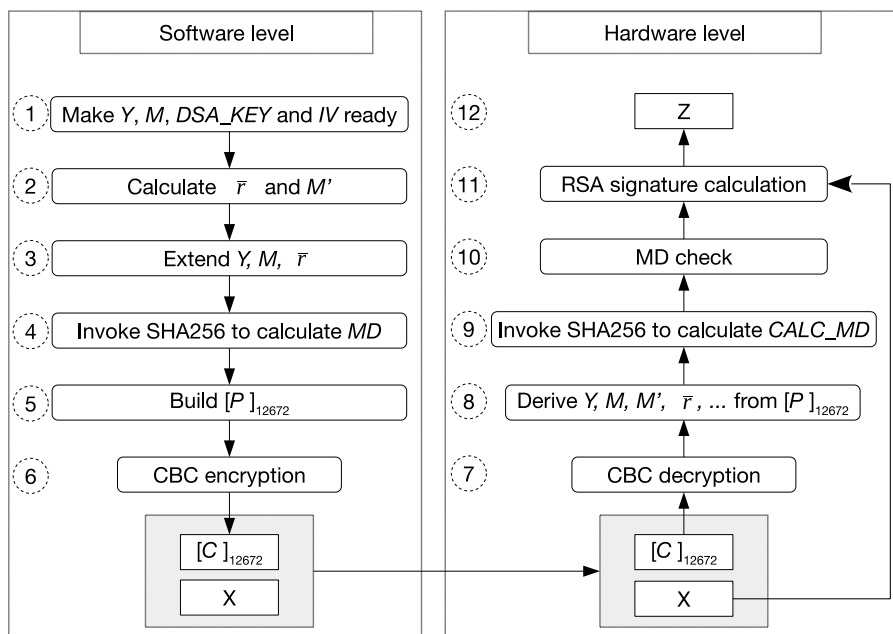


Figure 30.3-1. Software Preparations and Hardware Working Process

Note:

1. The software preparation (left side in Figure 30.3-1) is a one-time operation before any signature is calculated, while the hardware calculation (right side in Figure 30.3-1) repeats for every signature calculation.
2. Software preparation requires configuring the clock reset. For more information, please refer to Chapter 10 [Reset and Clock](#).

Users need to follow the steps shown in the left part of Figure 30.3-1 to calculate C . Detailed instructions are as follows:

- **Step 1:** Prepare operands Y and M whose lengths should meet the requirements in Section 30.3.2. Define $[L]_{32} = \frac{N}{32} - 1$ (i.e., for RSA 4096, $[L]_{32} = [0x80-1]_{32}$). Prepare $[RSA_DS_KEY]_{256}$:

- when calculated by the RSA_DS peripheral via the HMAC peripheral: $RSA_DS_KEY = \text{HMAC-SHA256}([HMAC_KEY]_{256}, 1^{256})$.
- when from Key Manager module: just use the deployed RSA_DS_KEY

Generate a random $[IV]_{128}$ which should meet the requirements of the AES-CBC block encryption algorithm. For more information on AES, please refer to Chapter 25 [AES Accelerator \(AES\)](#).

- **Step 2:** Calculate $\bar{r}$ and M' based on M .
- **Step 3:** Extend Y , M , and $\bar{r}$ in order to get $[Y]_{4096}$, $[M]_{4096}$, and $[\bar{r}]_{4096}$, respectively. This step is only required for Y , M , and $\bar{r}$ whose length are less than 4096 bits, since their largest length are 4096 bits.
- **Step 4:** Calculate MD authentication code using the SHA-256:
 $[MD]_{256} = \text{SHA256}([Y]_{4096} || [M]_{4096} || [\bar{r}]_{4096} || [M']_{32} || [L]_{32} || [IV]_{128})$
- **Step 5:** Build $[P]_{12672} = ([Y]_{4096} || [M]_{4096} || [\bar{r}]_{4096} || [Box]_{384})$, where $[Box]_{384} = ([MD]_{256} || [M']_{32} || [L]_{32} || [\beta]_{64})$ and $[\beta]_{64}$ is a PKCS#7 padding value, i.e., a $[0x0808080808080808]_{64}$ string composed of 8 bytes (0x80). The purpose of $[\beta]_{64}$ is to make the bit length of P a multiple of 128.

- **Step 6:** Calculate $C = [C]_{12672} = \text{AES-CBC-ENC}([P]_{12672}, [RSA_DS_KEY]_{256}, [IV]_{128})$, where C is the ciphertext with a length of 1200 bytes. C can also be calculated as $C = [C]_{12672} = ([\hat{Y}]_{4096} || [\hat{M}]_{4096} || [\hat{r}]_{4096} || [\hat{Box}]_{384})$, where $[\hat{Y}]_{4096}$, $[\hat{M}]_{4096}$, $[\hat{r}]_{4096}$, $[\hat{Box}]_{384}$ are the four sub-parameters of C , and correspond to the ciphertext of $[Y]_{4096}$, $[M]_{4096}$, $[r]_{4096}$, $[Box]_{384}$ respectively.

30.3.4 RSA_DS Operation at the Hardware Level

The hardware operation is triggered each time a digital signature needs to be calculated. The inputs are the pre-generated private key ciphertext C , a unique message X , and IV .

The RSA_DS operation at the hardware level can be divided into the following three stages:

1. Decryption: Step 7 and 8 in Figure 30.3-1

The decryption process is the inverse of Step 6 in Figure 30.3-1. The RSA_DS peripheral will call the AES accelerator to decrypt C in CBC block mode and get the resulting plaintext. The decryption process can be represented by $P = \text{AES-CBC-DEC}(C, RSA_DS_KEY, IV)$, where IV (i.e., $[IV]_{128}$) is defined by the user. $[RSA_DS_KEY]_{256}$ is

- when calculated by the RSA_DS peripheral via the HMAC peripheral: provided by the HMAC module, derived from $HMAC_KEY$ stored in eFuse, which is not readable by users.
- when from Key Manager module: RSA_DS_KEY deployed by Key Manager, which is not readable by users.

With P , the RSA_DS peripheral can derive $[Y]_{4096}$, $[M]_{4096}$, $[r]_{4096}$, $[M']_{32}$, $[L]_{32}$, MD authentication code, and the padding value $[\beta]_{64}$. This process is the inverse of Step 5.

2. Check: Step 9 and 10 in Figure 30.3-1

The RSA_DS peripheral will perform two checks: MD check and padding check. The padding check is not shown in Figure 30.3-1, as it happens at the same time as the MD check.

- MD check: The RSA_DS peripheral calls SHA-256 to calculate the hash value $[CALC_MD]_{256}$ ($[CALC_MD]_{256}$ is calculated the same way and with same parameters as $[MD]_{256}$, see step 4). Then, $[CALC_MD]_{256}$ is compared against the MD authentication code $[MD]_{256}$ from step 4. Only when the two match does the MD check pass.
- Padding check: The RSA_DS peripheral checks if $[\beta]_{64}$ complies with the aforementioned PKCS#7 format. Only when $[\beta]_{64}$ complies with the format does the padding check pass.

The RSA_DS peripheral will only perform subsequent operations if MD check passes. If the padding check fails, a warning is generated, but it does not affect the subsequent operations.

3. Calculation: Step 11 and 12 in Figure 30.3-1

The RSA_DS peripheral treats X (input by the user) and Y , M , $\bar{r}$ (decrypted in step 8) as big numbers. With M' , all operands to perform $X^Y \bmod M$ are in place. The operand length is defined by L only. The RSA_DS peripheral will calculate the signed result Z by calling RSA to perform $Z = X^Y \bmod M$.

30.3.5 RSA_DS Operation at the Software Level

The software steps below should be followed each time a digital signature needs to be calculated. The inputs are the pre-generated private key ciphertext C , a unique message X , and IV . These software steps trigger

the hardware steps described in Section 30.3.4.

We assume that the software has called the HMAC peripheral and the HMAC peripheral has calculated *RSA_DS_KEY* based on *HMAC_KEY*.

1. **Prerequisites:** Prepare operands *C*, *X*, *IV* according to Section 30.3.3.
2. **Activate the RSA_DS peripheral:** Write 1 to [DSA_SET_START_REG](#).
3. **Check if *RSA_DS_KEY* is ready** depending on from where the *RSA_DS_KEY* comes from:
 - Write 1 to [DSA_KEY_SOURCE_REG](#) to select *RSA_DS_KEY* deployed by the Key Manager. Verify that *RSA_DS_KEY* has been successfully deployed by checking the register field [KEYMNG_KEY_DS_VLD](#). When its value is 1, it indicates that *RSA_DS_KEY* has been successfully deployed and is ready.
 - Write 0 to [DSA_KEY_SOURCE_REG](#) to select *RSA_DS_KEY* from HMAC. Poll [DSA_QUERY_BUSY_REG](#) until the software reads 0.

If the software does not read 0 in [DSA_QUERY_BUSY_REG](#) after approximately 1 ms, it indicates a problem with HMAC initialization. In such a case, the software can read register [DSA_QUERY_KEY_WRONG_REG](#) to get more information:

- If the software reads 0 in [DSA_QUERY_KEY_WRONG_REG](#), it indicates that the HMAC peripheral has not been called.
 - If the software reads any value from 1 to 15 in [DSA_QUERY_KEY_WRONG_REG](#), it indicates that HMAC was called, but the RSA_DS peripheral did not successfully get *RSA_DS_KEY* from the HMAC peripheral. This may indicate that the HMAC operation has been interrupted due to a software concurrency problem.
4. **Configure register:** Write the content in the *IV* block to register [DSA_IV_m_REG](#) (*m*: 0 ~ 3). For more information on the *IV* block, please refer to Chapter 25 *AES Accelerator (AES)*.
 5. **Write *X* to memory block [DSA_X_MEM](#):** Write X_i ($i \in \{0, 1, \dots, n-1\}$), where $n = \frac{N}{32}$, to memory block [DSA_X_MEM](#) whose capacity is 96 words. Each word can store one base-*b* digit. The memory block uses the little endian format for storage, i.e., the least significant digit of the operand is in the lowest address. Words in [DSA_X_MEM](#) block after the configured length of *X* (*N* bits, as described in Section 30.3.2), are ignored.
 6. **Write *C* to corresponding memory blocks:** Write the four sub-parameters of *C* to corresponding memory blocks:
 - Write $\widehat{Y}_i$ ($i \in \{0, 1, \dots, 127\}$) to [DSA_Y_MEM](#).
 - Write $\widehat{M}_i$ ($i \in \{0, 1, \dots, 127\}$) to [DSA_M_MEM](#).
 - Write $\widehat{r}_i$ ($i \in \{0, 1, \dots, 127\}$) to [DSA_RB_MEM](#).
 - write $\widehat{Box}_i$ ($i \in \{0, 1, \dots, 11\}$) to [DSA_BOX_MEM](#).

The capacity of [DSA_Y_MEM](#), [DSA_M_MEM](#), and [DSA_RB_MEM](#) is 128 words, whereas the capacity of [DSA_BOX_MEM](#) is only 12 words. Each word can store one base-*b* digit. The memory blocks use the little endian format for storage, i.e., the least significant digit of the operand is in the lowest address.

7. **Start RSA_DS operation:** Write 1 to register [DSA_SET_ME_REG](#).

8. **Wait for the operation to be completed:** Poll register [DSA_QUERY_BUSY_REG](#) until the software reads 0.
9. **Query check result:** Read register [DSA_QUERY_CHECK_REG](#) and conduct subsequent operations as illustrated below based on the return value:
 - If the value is 0, it indicates that both the padding check and MD check pass. Users can continue to get the signed result Z .
 - If the value is 1, it indicates that the padding check passes but MD check fails. The signed result Z is invalid. The operation will resume directly from Step 11.
 - If the value is 2, it indicates that the padding check fails but the MD check passes. Users can continue to get the signed result Z . But please note that the data does not comply with the aforementioned PKCS#7 padding format, which may not be what you want.
 - If the value is 3, it indicates that both the padding check and MD check fail. In this case, some fatal errors have occurred and the signed result Z is invalid. The operation will resume directly from Step 11.
10. **Read the signed result:** Read the signed result Z_i ($i \in \{0, 1, \dots, n - 1\}$), where $n = \frac{N}{32}$, from memory block [DSA_Z_MEM](#). The memory block stores Z in little-endian byte order.
11. **Exit the operation:** Write 1 to [DSA_SET_FINISH_REG](#), and then poll [DSA_QUERY_BUSY_REG](#) until the software reads 0.

After the operation, all the input/output registers and memory blocks are cleared.

30.4 Memory Summary

The addresses in this section are relative to the RSA Digital Signature Peripheral (RSA_DS) base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

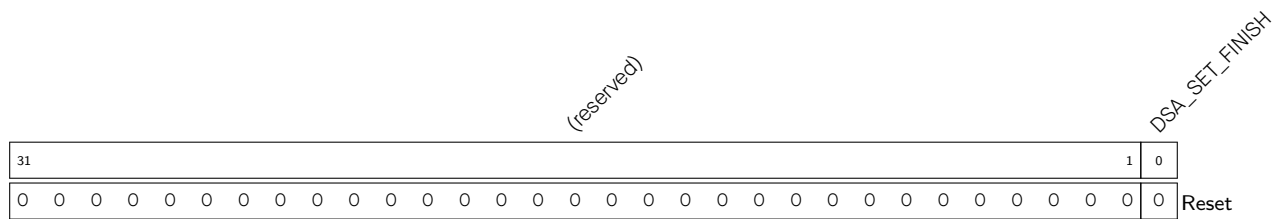
Name	Description	Size (byte)	Starting Address	Ending Address	Access
DSA_Y_MEM	Memory block Y	512	0x0000	0x01FF	WO
DSA_M_MEM	Memory block M	512	0x0200	0x03FF	WO
DSA_RB_MEM	Memory block $\bar{r}$	512	0x0400	0x05FF	WO
DSA_BOX_MEM	Memory block Box	48	0x0600	0x062F	WO
DSA_X_MEM	Memory block X	512	0x0800	0x09FF	WO
DSA_Z_MEM	Memory block Z	512	0x0A00	0x0BFF	RO

30.5 Register Summary

The addresses in this section are relative to the RSA Digital Signature Peripheral (RSA_DS) base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Configuration Registers			
DSA_IV_0_REG	IV block data	0x0630	WO
DSA_IV_1_REG	IV block data	0x0634	WO
DSA_IV_2_REG	IV block data	0x0638	WO
DSA_IV_3_REG	IV block data	0x063C	WO
DSA_KEY_SOURCE_REG	Configures RSA_DS_KEY source	0x0E18	R/W
Status/Control Registers			
DSA_SET_START_REG	Activates the RSA_DS peripheral	0x0E00	WO
DSA_SET_ME_REG	Starts RSA_DS operation	0x0E04	WO
DSA_SET_FINISH_REG	Ends RSA_DS operation	0x0E08	WO
DSA_QUERY_BUSY_REG	Status of the RSA_DS peripheral	0x0E0C	RO
DSA_QUERY_KEY_WRONG_REG	Checks the reason why <i>RSA_DS_KEY</i> is not ready	0x0E10	RO
DSA_QUERY_CHECK_REG	Queries DSA check result	0x0E14	RO
Version control register			
DSA_DATE_REG	Version control register	0x0E20	W/R

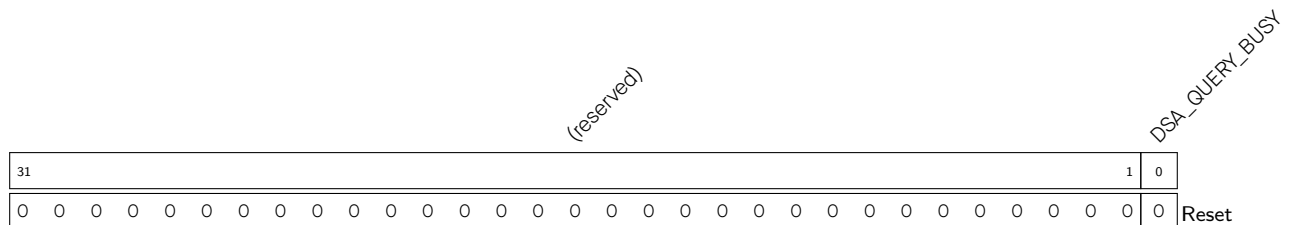
Register 30.4. DSA_SET_FINISH_REG (0x0E08)

DSA_SET_FINISH Configures whether to end the RSA_DS operation.

0: No effect

1: End the RSA_DS operation

(WO)

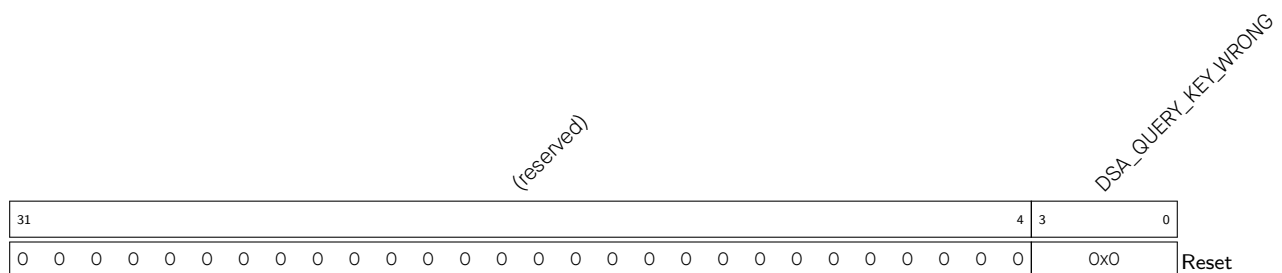
Register 30.5. DSA_QUERY_BUSY_REG (0x0E0C)

DSA_QUERY_BUSY Represents whether the RSA_DS peripheral is idle.

0: The RSA_DS peripheral is idle.

1: The RSA_DS peripheral is busy.

(RO)

Register 30.6. DSA_QUERY_KEY_WRONG_REG (0x0E10)

DSA_QUERY_KEY_WRONG Represents the specific problem with HMAC initialization.

0: HMAC is not called.

1-15: HMAC was activated, but the RSA_DS peripheral did not successfully receive *RSA_DS_KEY* from the HMAC peripheral.

(RO)

Register 30.7. DSA_QUERY_CHECK_REG (0x0E14)

(reserved)																															DSA_PADDING_BAD DSA_MD_ERROR																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
31																															2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

DSA_PADDING_BAD Represents whether the padding check passes.

0: The padding check passes.

1: The padding check fails.

(RO)

DSA_MD_ERROR Represents whether the MD check passes.

0: The MD check passes.

1: The MD check fails.

(RO)

Register 30.8. DSA_KEY_SOURCE_REG (0x0E18)

(reserved)																															DS_KEY_SOURCE																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																
31																															1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

DSA_KEY_SOURCE Represents the source of *RSA_DS_KEY*.

0: HMAC

1: Key Manager

(R/W)

Register 30.9. DSA_DATE_REG (0x0E20)

(reserved)																															DSA_DATE																														
31	30	29																																																								0			
0	0	0x20200618																																																							Reset				

DSA_DATE Version control register. (R/W)